

Phase 1 Upgrade Plan — Repo Structure, Automated Data Collection, and Local Database Setup

Objective

Upgrade the initial Alpaca API test into a clean, organized, scalable local research platform.

This phase focuses on:

1. Professional GitHub repository structure
2. Multi-file Python project organization
3. Automated market data collection
4. Local database storage (before cloud migration)
5. Team collaboration readiness
6. Clean development workflow

The goal is to build a stable foundation before adding:

- AI layers
- options flow
- cloud infrastructure
- automation complexity

HIGH-LEVEL TARGET ARCHITECTURE

```
Alpaca API
  ↓
market_fetcher.py
  ↓
storage.py
  ↓
DuckDB Local Database
  ↓
analysis.py
  ↓
future AI layer
```

PART 1 — CLEAN REPOSITORY STRUCTURE

Recommended Repository Layout

```
family-quant-ai/
|
├─ README.md
├─ .gitignore
├─ requirements.txt
├─ .env
|
├─ data/
|   ├── raw/
|   ├── processed/
|   └── databases/
|       └── market_data.duckdb
|
├─ scripts/
|   ├── market_fetcher.py
|   ├── storage.py
|   ├── analysis.py
|   └── scheduler_test.py
|
├─ notebooks/
|   └── exploration.ipynb
|
├─ docs/
|   ├── api_notes.md
|   ├── architecture.md
|   └── lessons_learned.md
|
├─ logs/
|   └── collector.log
|
└─ tests/
    └── test_connection.py
```

PURPOSE OF EACH FOLDER

Folder	Purpose
data/raw	original downloaded market data
data/processed	cleaned/transformed data
data/databases	DuckDB database files
scripts	actual Python logic
notebooks	experimentation and learning
docs	documentation and architecture
logs	execution logs/errors
tests	validation scripts

PART 2 — GITHUB CLEANUP & BEST PRACTICES

Recommended .gitignore

```
venv/  
__pycache__/  
.env  
*.pyc  
.ipynb_checkpoints/  
logs/  
.DS_Store
```

README.md Structure

Recommended Sections

```
# Family Quant AI Lab  
  
## Overview  
Learning-focused AI-assisted market research platform.  
  
## Current Phase
```

Phase 1 – Market Data Collection

Features

- Alpaca API integration
- Local DuckDB storage
- Automated data collection
- Historical market tracking

Tech Stack

- Python
- DuckDB
- Alpaca API
- VS Code

Run Instructions

```
python scripts/market_fetcher.py
```

PART 3 — LOCAL DATABASE SETUP (IMPORTANT)

Why Start Local First?

Starting locally is strongly recommended because:

- easier debugging
- lower cost
- less infrastructure complexity
- better learning experience
- faster iteration
- easier collaboration with Benjamin

Cloud comes later.

Why DuckDB?

Recommended Database

Use:

DuckDB

Why:

- extremely fast
- very beginner friendly
- no server setup
- ideal for analytical workloads
- excellent for finance data
- single-file database
- integrates well with Python

Think of it as:

“SQLite for analytics and quant research”

Install DuckDB

Inside activated venv:

```
python -m pip install duckdb pandas
```

Database File Location

Recommended:

```
/data/databases/market_data.duckdb
```

This single file becomes:

- your historical market archive
 - your analytics foundation
 - your future AI training source
-

PART 4 — CREATE YOUR FIRST REAL MARKET FETCHER

Rename Existing Script

Rename:

test_alpaca.py

To:

market_fetcher.py

Move into:

/scripts/

Recommended market_fetcher.py Responsibilities

This file should:

1. Connect to Alpaca API
2. Download market data
3. Parse response
4. Save to DuckDB
5. Log success/errors

NOT:

- analysis
- AI
- trading logic

Keep responsibilities separated.

PART 5 — CREATE storage.py

Purpose

This file manages:

- database connection
- table creation
- inserts
- queries

This separation is VERY important.

Example Responsibilities

storage.py

```
connect_database()
create_tables()
save_stock_data()
load_historical_data()
```

PART 6 — FIRST DATABASE TABLE DESIGN

Recommended Initial Table

Table:

```
stock_prices
```

Columns:

Column	Type
timestamp	TIMESTAMP
symbol	VARCHAR
open	DOUBLE

Column	Type
high	DOUBLE
low	DOUBLE
close	DOUBLE
volume	BIGINT

Why This Structure Matters

This allows future:

- backtesting
- AI analysis
- signal detection
- charting
- strategy simulation

PART 7 — AUTOMATED DATA COLLECTION

Goal

Turn the manual script into:

a scheduled data collection system

Recommended Schedule (Phase 1)

Run:

- once daily after market close OR
- every hour during market hours

Keep it simple.

Windows Automation Recommendation

Use:

Windows Task Scheduler

NOT:

- Kubernetes
- Docker orchestration
- Airflow
- cloud schedulers

Too much complexity early.

Task Scheduler Flow

```
Windows Scheduler
  ↓
market_fetcher.py
  ↓
DuckDB
  ↓
log file update
```

Recommended First Watchlist

Start with:

Symbol	Purpose
SPY	overall market
QQQ	tech index
AAPL	large-cap tech
NVDA	AI-related volatility
TSLA	momentum/volatility

PART 8 — CREATE analysis.py

Goal

Do NOT add AI yet.

First build:

rule-based analysis.

Recommended First Signals

Signal 1

Relative volume spike.

Signal 2

Large daily price move.

Signal 3

New 20-day high.

Why No AI Yet?

Because:

- rule-based systems teach market logic
- easier debugging
- easier validation
- AI hides errors early

AI comes AFTER stable data pipelines.

PART 9 — DAILY WORKFLOW FOR YOU + BENJAMIN

Parent Responsibilities

- architecture
- organization
- documentation
- testing outputs
- reviewing logs
- project management

Benjamin Responsibilities

- coding
- debugging
- experimenting
- improving scripts
- testing new features

Recommended Daily Routine

1. Pull latest GitHub changes
2. Run `market_fetcher.py`
3. Verify data stored correctly
4. Commit improvements
5. Update Notion learning journal

PART 10 — GITHUB WORKFLOW

Recommended Commit Style

Small commits only.

Examples:

```
Add DuckDB connection
Add market data storage logic
```

Fix timestamp parsing
Add SPY watchlist support

Avoid giant commits.

PART 11 — MILESTONES

Milestone 1

Successfully fetch market data.

Status: ACHIEVED

Milestone 2

Store market data into DuckDB.

Target: next major goal

Milestone 3

Automate scheduled collection.

Milestone 4

Create historical query capability.

Milestone 5

Generate first rule-based market signals.

PART 12 — WHAT NOT TO DO YET

Avoid:

- cloud migration

- GPU rentals
- options flow APIs
- reinforcement learning
- trading bots
- automated execution
- real money trading
- multi-agent systems

These come MUCH later.

PART 13 — FUTURE CLOUD MIGRATION PLAN

ONLY move to cloud when:

- local storage grows significantly
- laptop becomes bottleneck
- need always-on collectors
- collaboration expands
- AI workloads increase

Recommended Future Cloud Architecture

```
graph TD; Laptop --> GitHub; GitHub --> Cloud_VPS[Cloud VPS]; Cloud_VPS --> Cloud_Storage[Cloud Storage]; Cloud_Storage --> Optional_GPU[Optional GPU];
```

FINAL RECOMMENDATION

Your current setup is already enough to build a surprisingly capable market research platform.

The most important thing now is:

- consistency

- organization
- documentation
- clean architecture
- gradual progression

The local DuckDB phase is one of the smartest choices you can make because it gives you:

- real-world data engineering skills
- fast analytics
- low complexity
- zero cloud cost
- strong foundation for future AI work

Build slowly and cleanly.